Data Cleaning & Quality Checks

This notebook performs data cleaning and validation on the raw datasets:

- customers
- policies
- claims
- payments

Objectives

1. Standardize data types (especially date columns)
2. Handle missing values
3. Identify and fix logical inconsistencies
4. Address outliers
5. Prepare clean datasets for analysis

This notebook builds on findings from the Initial Data Assessment notebook.

Imports and Loading Raw Data

In [1]:
```
#importing libraries
import pandas as pd
import numpy as np
import os

#loading raw datasets
customers = pd.read_csv("data/raw/customers.csv")
policies = pd.read_csv("data/raw/policies.csv")
claims = pd.read_csv("data/raw/claims.csv")
payments = pd.read_csv("data/raw/payments.csv")

#verifying loading
print("Customers shape:", customers.shape)
print("Policies shape:", policies.shape)
print("Claims shape:", claims.shape)
print("Payments shape:", payments.shape)
```

```
Customers shape: (1050, 5)
Policies shape: (1500, 6)
Claims shape: (708, 5)
Payments shape: (18000, 4)
```

Standardizing Date Columns

- Mixed date formats are standardized using `format='mixed'` with `dayfirst=True` to ensure consistent parsing across both ISO (`YYYY-MM-DD`) and regional (`DD/MM/YYYY`) formats.
- Invalid or unparseable dates are converted to `NaT`
- This step is required before performing logical checks

In [2]:
```
# Converting all date columns to datetime using mixed format handling
customers['join_date'] = pd.to_datetime(customers['join_date'], format = 'mixed', d
policies['start_date'] = pd.to_datetime(policies['start_date'], format = 'mixed', d
policies['end_date'] = pd.to_datetime(policies['end_date'], format = 'mixed', dayfi
claims['claim_date'] = pd.to_datetime(claims['claim_date'], format = 'mixed', dayfi
payments['payment_date'] = pd.to_datetime(payments['payment_date'], format = 'mixed
```

In [3]:
```
# Checking for date parsing errors
print("Unparsed join_date:", customers['join_date'].isna().sum())
print("Unparsed start_date:", policies['start_date'].isna().sum())
print("Unparsed end_date:", policies['end_date'].isna().sum())
print("Unparsed claim_date:", claims['claim_date'].isna().sum())
print("Unparsed payment_date:", payments['payment_date'].isna().sum())
```

```
Unparsed join_date: 0
Unparsed start_date: 0
Unparsed end_date: 0
Unparsed claim_date: 0
Unparsed payment_date: 0
```

Customers Table Cleaning

- Missing income values are filled with the median for simplicity
- Duplicate rows are removed

In [4]:
```
# Filling missing income with median
customers['income'] = customers['income'].fillna(customers['income'].median())

# Removing duplicate rows
customers = customers.drop_duplicates()

print(customers.info())
print(customers.duplicated().sum())
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 1000 entries, 0 to 999
Data columns (total 5 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   customer_id  1000 non-null   int64
 1   age          1000 non-null   int64
 2   region       1000 non-null   object
 3   income       1000 non-null   float64
 4   join_date    1000 non-null   datetime64[ns]
dtypes: datetime64[ns](1), float64(1), int64(2), object(1)
memory usage: 46.9+ KB
None
0
```

Policies Table Cleaning

- Checks are performed for logical issues with dates
- No policies have end dates before start dates
- Policies that start before customer join date are removed
- The policy_type column is formatted consistently

Logical Checks

In [5]:
```
# Merging with customers to access join_date
policies_merged = policies.merge(customers[['customer_id', 'join_date']], on = 'cus

# Check 1: start_date > end_date
invalid_dates = policies[policies['start_date'] > policies['end_date']]
print(f"policies with start_date after end_date: {invalid_dates.shape[0]}")

# Check 2: policie starts before customer joins
invalid_customer_dates = policies_merged[policies_merged['start_date'] < policies_m
print(f"Policies starting before customer join date: {invalid_customer_dates.shape[
```

```
policies with start_date after end_date: 0
Policies starting before customer join date: 150
```

In [6]:
```
# Removing policies before customer join date
policies = policies.merge(customers[['customer_id', 'join_date']], on = 'customer_i
policies = policies[policies['start_date'] >= policies['join_date']]

# Dropping helper column
policies = policies.drop(columns = ['join_date'])
```

In [7]:
```
# Standardizing policy_type formatting
policies['policy_type'].value_counts()
```

Out[7]:
```
policy_type
Auto    462
Life    367
Home    345
AUTO     98
auto     78
Name: count, dtype: int64
```

In [8]:
```
policies['policy_type'] = policies['policy_type'].str.strip()
policies['policy_type'] = policies['policy_type'].str.lower()

policies['policy_type'] = policies['policy_type'].replace({
    'auto': 'Auto',
    'home': 'Home',
    'life': 'Life'
})

policies['policy_type'].value_counts()
```

Out[8]:
```
policy_type
Auto    638
Life    367
Home    345
Name: count, dtype: int64
```

Claims Table Cleaning

- Missing claim types are filled.
- Negative claim amounts are removed rather than converted to positive values.
- These values likely represent data entry errors, reversals, or adjustments rather than valid claims. Converting them to positive would introduce incorrect financial data and distort analysis results. In a production system these would be reviewed rather than removed.
- Extreme outliers are capped at 99th percentile excluding the max value
- Some claims have invalid policy ids. These are dropped.

In [9]:
```
# Filling missing claim_type
claims['claim_type'] = claims['claim_type'].fillna('Unknown')

# Removing negative claim amounts
claims = claims[claims['claim_amount'] >= 0]

# Keep only claims with valid policy_id
claims = claims[claims['policy_id'].isin(policies['policy_id'])]
print(f"Claims after fixing policy_id: {claims.shape[0]}")

# Capping extreme outliers
upper_limit = claims['claim_amount'][claims['claim_amount'] != claims['claim_amount
claims['claim_amount'] = claims['claim_amount'].clip(upper=upper_limit)

print("Max after capping:", claims['claim_amount'].max())
```

```
Claims after fixing policy_id: 628
Max after capping: 6109.776100000007
```

Payments Table Cleaning

- Invalid payment amounts are removed

In [10]:
```python
# Removing negative or zero payments (if any)
payments = payments[payments['payment_amount'] > 0]
print(payments.describe())
```

```
          policy_id  payment_amount                payment_date    payment_id
count  18000.000000    18000.000000                       18000  18000.000000
mean     750.500000      145.270444  2021-03-24 20:08:19.200000   9000.500000
min        1.000000       41.833333         2018-01-01 00:00:00      1.000000
25%      375.750000       94.291667         2019-11-01 00:00:00   4500.750000
50%      750.500000      144.500000         2021-03-04 00:00:00   9000.500000
75%     1125.250000      196.416667         2022-09-06 06:00:00  13500.250000
max     1500.000000      249.583333         2024-06-05 00:00:00  18000.000000
std      433.024634       59.631568                         NaN   5196.296758
```

Final Validation Checks

After all cleaning steps, final validation is performed to ensure data integrity:

1. Confirming the **shape** of each table (number of rows and columns) matches expectations.
2. Checking for **missing values** — all critical fields should be filled or handled.
3. Checking for **duplicate rows** — ensure there are no accidental duplicates.

In [11]:
```python
# Customers table
print("CUSTOMERS TABLE")
print("-------------------")
print("Shape:", customers.shape)
print("Missing values:\n", customers.isna().sum())
print("Duplicate rows:", customers.duplicated().sum())
print("\n")

# Policies table
print("POLICIES TABLE")
print("-------------------")
print("Shape:", policies.shape)
print("Missing values:\n", policies.isna().sum())
print("Duplicate rows:", policies.duplicated().sum())
print("\n")

# Claims table
print("CLAIMS TABLE")
print("-------------------")
print("Shape:", claims.shape)
print("Missing values:\n", claims.isna().sum())
print("Duplicate rows:", claims.duplicated().sum())
```

```
print("\n")

# Payments table
print("PAYMENTS TABLE")
print("-------------------")
print("Shape:", payments.shape)
print("Missing values:\n", payments.isna().sum())
print("Duplicate rows:", payments.duplicated().sum())
print("\n")
```

```
CUSTOMERS TABLE
-------------------
Shape: (1000, 5)
Missing values:
 customer_id    0
age            0
region         0
income         0
join_date      0
dtype: int64
Duplicate rows: 0


POLICIES TABLE
-------------------
Shape: (1350, 6)
Missing values:
 customer_id       0
policy_type       0
premium_amount    0
policy_id         0
start_date        0
end_date          0
dtype: int64
Duplicate rows: 0


CLAIMS TABLE
-------------------
Shape: (628, 5)
Missing values:
 policy_id       0
claim_amount    0
claim_date      0
claim_type      0
claim_id        0
dtype: int64
Duplicate rows: 0


PAYMENTS TABLE
-------------------
Shape: (18000, 4)
Missing values:
 policy_id         0
payment_amount    0
payment_date      0
payment_id        0
dtype: int64
Duplicate rows: 0
```

Exporting Clean Data

In [12]:
```
os.makedirs("data/clean", exist_ok = True)

customers.to_csv("data/clean/customers_clean.csv", index = False)
policies.to_csv("data/clean/policies_clean.csv", index = False)
claims.to_csv("data/clean/claims_clean.csv", index = False)
payments.to_csv("data/clean/payments_clean.csv", index = False)
```

Final Notes

The datasets are now:

- Cleaned and standardized
- Free of major logical inconsistencies
- Ready for analysis or visualization

These cleaned datasets will be used in downstream analysis notebooks.

In [ ]: